

Servidor simulado con asignador de recursos

Sistemas Operativos · TP Obligatorio · Síntesis de la propuesta

La idea

La máquina de Red Hat simula ser el **servidor central de una empresa ficticia** (por ejemplo, "PagoSur", una fintech). Cuando se incorpora un departamento o proyecto nuevo, el administrador ejecuta un asignador que le crea usuarios, espacio en disco con cuota, límites de CPU y memoria, permisos y reglas de seguridad, dejando todo registrado en un log de auditoría.

```
./asignar.sh --depto finanzas --usuarios finanzas.csv \  
--disco 2G --cpu 30% --ram 512M
```

El servidor es el **escenario común**, pero la consigna exige 6 ejercicios de unidades distintas. Por eso cada pieza del asignador es un ejercicio independiente, con su propio enunciado, solución con capturas y apartado teórico.

Los 6 ejercicios

Ejercicio	Qué hace	Herramientas Red Hat	Código	Teoría
1. Estructura del SO	Inventario del servidor: reporta los recursos totales disponibles para repartir.	/proc/cpuinfo, /proc/meminfo, strace, lscpu	C	Syscalls, modo usuario/kernel, /proc
2. Procesos	Monitor que detecta procesos que exceden lo asignado y los baja de prioridad o los termina.	ps, top, pstree, renice, kill, /proc/<PID>/stat	C o script	Estados, PCB, señales, zombies
3. Planificación de CPU	Asigna % de CPU por departamento con slices de systemd (cgroups).	systemctl set-property ... CPUQuota=30%, top, nice, chrt	C (carga de prueba)	Algoritmos de scheduling, CFS
4. Memoria	Límite de RAM por departamento; se demuestra el OOM killer sin afectar al resto.	MemoryMax, free, vmstat, pmap, journalctl, swap	C (malloc en loop)	Memoria virtual, paginación, swapping
5. Almacenamiento	Un volumen lógico por departamento, montaje persistente y cuotas por usuario; extensión a demanda.	fdisk, pvcreate, vgcreate, lvcreate, lvextend, mkfs.xfs, fstab, xfs_quota	Script	Sistemas de archivos, inodos, journaling
6. Seguridad	Alta de usuarios desde CSV, permisos por rol, auditor de solo lectura, sudo limitado, SELinux.	useradd, groupadd, chage, chmod, SGID, setfacl, sudoers, semanage, restorecon	Script	DAC vs MAC, mínimo privilegio

Arquitectura del asignador

El script principal es corto y solo llama a módulos, uno por ejercicio. Así cada pieza se puede mostrar y defender por separado, y cualquier integrante puede explicar cualquier parte.

```
asignar.sh          # orquestador: valida parámetros y llama a los módulos
modulos/
  01_inventario.c   # recursos disponibles
  02_monitor.c      # control de procesos
  03_cpu.sh         # slice + CPUQuota
  04_memoria.sh     # MemoryMax
  05_almacenamiento.sh # LVM + XFS + cuotas
  06_seguridad.sh   # usuarios, permisos, ACL, SELinux
logs/asignador.log  # auditoría de cada acción
```

Cumplimiento de la consigna

6 ejercicios, uno por unidad	Sí, cada módulo es un ejercicio con su unidad.
Consola de Red Hat en todos	Sí, todos se resuelven y demuestran por terminal.
Shell scripting en al menos 3	Sí: ejercicios 5, 6 y el orquestador (3 y 4 también).
C/C++ (valorado)	Sí: ejercicios 1, 2, 3 y 4.
Enunciado hipotético creativo	Escenario de empresa único que da coherencia al TP.
Apartado teórico por ejercicio	Columna "Teoría" de la tabla como punto de partida.

Consideraciones técnicas

- **Discos:** si la VM no tiene discos extra, se simulan con archivos y loop devices (dd + losetup) y se arma el volume group encima.
- **Cuotas en XFS:** el volumen debe montarse con uquota o pquota; si no, xfs_quota no tiene efecto.
- **Entorno del lab:** verificar si se resetea entre sesiones y guardar todo en un repositorio para reconstruir rápido.
- **Independencia:** cada ejercicio debe poder demostrarse solo, para que un error en uno no rompa la demo de los demás.
- **Defensa:** todos deben entender los 6 módulos; conviene rotar quién prepara cada uno.